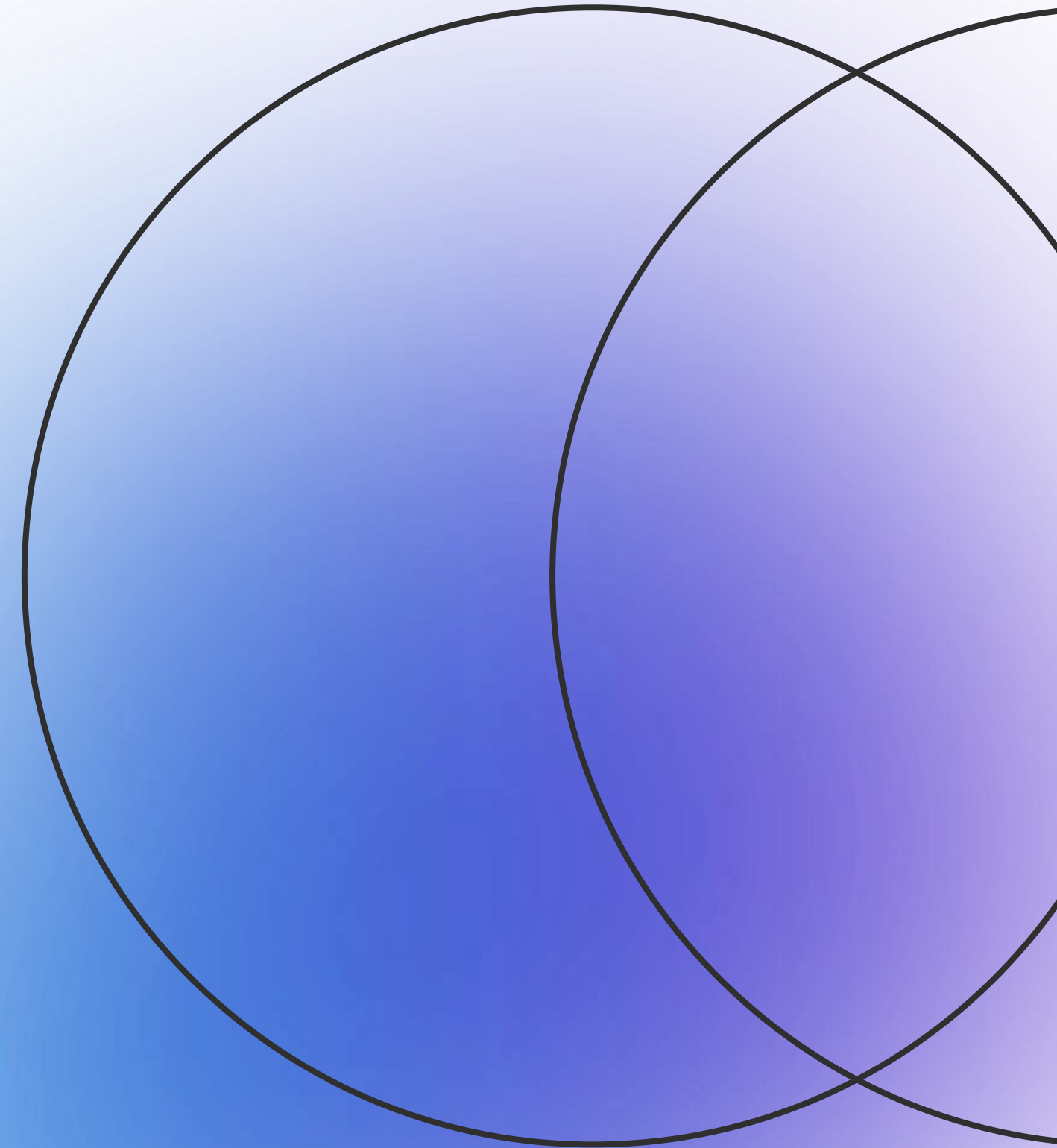


Dominic Wooldridge

Basis

An agentic IRA tax consultant

GSB 570 - AI for Bus Analytics



Context / The Problem

The Inflation Reduction Act (2022) is a federal law that offers tax credits worth up to 50% of project costs to developers who build clean energy infrastructure in the United States.

The Inflation Reduction Act introduced the most significant clean energy tax incentives in US history — but qualifying for the maximum credit requires navigating four overlapping rule sets simultaneously:

- **Base rate:** 6% ITC, jumping to 30% only if you meet prevailing wage AND apprenticeship requirements
- **Bonus adders:** +10% for energy community, +10% for domestic content, +10–20% for low-income — each with its own eligibility test
- **ITC vs. PTC election:** developers must model both and pick the one with higher NPV
- **Rules change:** IRS and Treasury issue new notices regularly; what qualified last quarter may not qualify today

A developer with a 100 MW solar project in the wrong county, or who misses a prevailing wage filing, can leave \$20–30M on the table. Today that analysis is done manually by expensive tax counsel — if it gets done at all.

What Basis Does

An AI pipeline that runs a full IRA credit analysis in under a minute

A developer submits their project specs — technology, location, capacity, compliance status — and Basis returns a structured investment memo showing:

- Which credits they qualify for and at what rate
- Whether their site is in a designated energy community
- IRR and NPV in two scenarios: with IRA credits and without
- The dollar value of the credit on their specific project
- Risk flags and a plain-English recommendation

What used to take a tax attorney and a financial analyst two days now takes 45 seconds.

Tech Stack

Built on four layers

Layer	Technology
LLM	Claude claude-sonnet-4-5 via AWS Bedrock
Agent framework	LangChain — structured outputs via tool use
Backend	Python, FastAPI, Pydantic v2
Financial model	Pure numpy DCF — no LLM touches the math
Data	NREL PVWatts API, EIA electricity prices, NETL energy community shapefiles
Web search	DuckDuckGo via LangChain Community
Frontend	React, Vite, Tailwind CSS
Infrastructure	AWS Bedrock (SSO auth), FastAPI local / Lambda target

Key design constraint:

every agent returns a validated Pydantic object. Raw LLM text never passes between agents. Numbers are always computed in Python — the LLM reasons and writes, not calculates.

Why this stack

Claude (Bedrock) — the only model that reliably interprets layered IRA statutory language and returns structured output, not prose. Bedrock gives us SSO access through the class AWS environment.

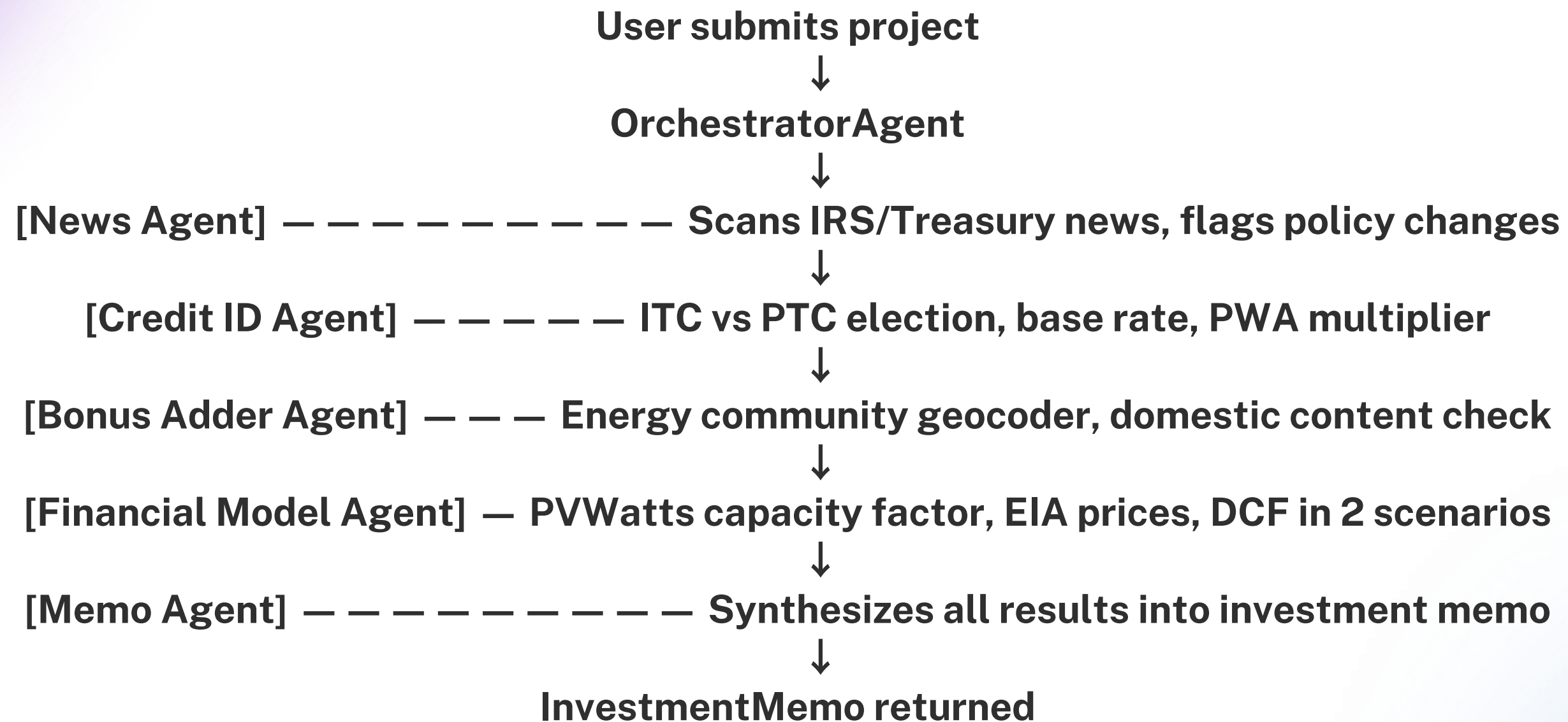
LangChain structured outputs — forces every agent to return a validated Pydantic object via Claude's tool-use API. No string parsing, no hallucinated numbers passing between agents.

FastAPI + Pydantic — the same schemas that validate agent outputs also validate the API. One type system across the entire backend.

numpy for all math — IRR, NPV, and payback are computed in Python, not by the LLM. The model reasons; Python calculates. That separation is what makes the financial output trustworthy.

React + Tailwind — fast to build, easy to demo.

Agentic Pipeline



Each agent has one job and one output schema. The orchestrator passes validated objects between them — not strings, not JSON blobs. If an agent fails its schema validation, the pipeline stops before bad data propagates downstream.

Why this Pipeline

Single-prompt approaches fail on IRA analysis because the reasoning has four distinct stages — regulatory interpretation, geographic lookup, financial math, and narrative synthesis — each requiring a different knowledge type. Forcing one LLM call to do all four simultaneously makes errors invisible and outputs unauditable.

Decomposing the work means each stage can be validated independently. The geocoder either places the project inside an energy community boundary or it doesn't. The DCF either returns a credible IRR or it doesn't. By the time the Memo Agent writes the executive summary, every input it receives is a typed, validated Python object — the facts are locked in before the model writes a single word.

Under the Hood — How the Pipeline Actually Runs

Every agent follows the same pattern: build a prompt in Python → send to Claude → force the response through a typed schema → pass the object to the next agent.

The key mechanism is LangChain's `.with_structured_output()`, which sends each agent's Pydantic model as a tool definition to Claude's API. Claude must respond by populating that schema with valid typed values — it cannot return free text. If any field fails validation, the pipeline stops.

Under the Hood — How the Pipeline Actually Runs

What actually happens at each step:

- **News Agent** — fires 3 DuckDuckGo searches in parallel via `asyncio.gather()`, concatenates the results, asks Claude to identify anything materially relevant to this project type
- **Credit Agent** — embeds the IRA rules explicitly in the prompt so Claude reasons against a stated ruleset, not training-time memory
- **Bonus Adder Agent** — runs a shapely point-in-polygon lookup against NETL's GeoJSON shapefiles before touching the LLM; the geocoder result is a hard boolean the model cannot override
- **Financial Agent** — calls NREL PVWatts and the EIA API in Python, runs IRR/NPV/payback via `numpy_financial`, then hands Claude only the analyst note to write; the model never touches the math
- **Memo Agent** — receives all four prior outputs as typed Python objects and writes the narrative; FastAPI serializes the result directly to the JSON response

The orchestrator itself contains no LLM and no logic — it calls each agent in sequence, passes the output forward, and returns the final memo.

The News Agent

Job: Patch the gap between the model's training cutoff and today's rules.

Claude's knowledge of IRA regulations is frozen at its training date. Treasury and IRS have issued dozens of notices since the Act passed. The News Agent runs three parallel web searches before every analysis — IRA policy updates, credit rule changes for the specific technology and state, and new IRS/Treasury notices.

The LLM reads the raw results and answers one question: does anything here materially change what this project qualifies for?

The output is a 2-3 sentence regulatory note that flows into every downstream agent as live context. If a new notice updated the energy community boundary last month, the next analysis knows.

The Credit Identification Agent

Job: Determine the base credit rate and recommend ITC vs. PTC.

Decision 1 — The PWA multiplier The base ITC rate is 6%. A 5x multiplier applies — bringing it to 30% — only if the developer meets both prevailing wage and apprenticeship requirements. One without the other leaves the rate at 6%. The agent checks each condition independently.

Decision 2 — ITC vs. PTC election Developers elect one credit type per project. The agent reasons through the tradeoff:

- **ITC (48E):** one-time credit on CapEx, taken at placed-in-service. Better for capital-intensive projects.
- **PTC (45Y):** per-kWh production credit over 10 years. Better for high-capacity-factor projects

The agent recommends whichever structure produces higher NPV given the project's technology and location.

Bonus Adder Agent

Job: Check three bonus adders and stack them onto the base rate.

Energy community (+10%) Not an LLM judgment — a geographic fact. The geocoder runs a point-in-polygon lookup against NETL's official shapefiles of coal closure communities and fossil fuel MSAs. The shapefile returns a boolean. The LLM stacks the adder if it's true.

Domestic content (+10%) Compares the developer's self-reported domestic content percentage against the statutory threshold: 40% for solar and storage, 20% for wind. If the threshold is met, the adder applies. If the number is close to the cutoff, it becomes a risk flag.

Low-income community (+10% or +20%) Requires an IRS allocation from a limited annual pool. The agent notes eligibility but does not assume the allocation has been granted.

Final output: stacked rate = base rate + all applicable adders. Maximum ~50%.

Financial Model Agent

Job: Run the numbers. No LLM involved in the math.
This agent is different from the others — everything is computed in Python.

Input	Source
Capacity factor	NREL PVWatts API (solar) or ATB regional table (wind, other)
Electricity price	EIA API — most recent commercial retail rate for the state
CapEx	Developer-provided
ITC credit value	$\text{stacked_rate} \times \text{capex}$

The DCF runs twice — once with the ITC applied as a year-0 offset against CapEx, once without. Both scenarios return IRR, NPV, and payback period.

The LLM's only role is writing a two-sentence analyst note. Every number is deterministic. The model cannot hallucinate a more attractive IRR.

Memo Agent

Job: Synthesize all four prior outputs into a structured investment memo.

The Memo Agent is the only one with visibility into the full picture. It receives:

- **Base rate and ITC/PTC recommendation (Credit Agent)**
- **Stacked rate and which adders applied (Bonus Adder Agent)**
- **IRR, NPV, payback in both scenarios (Financial Model Agent)**
- **Current regulatory context (News Agent)**

It generates four written sections — executive summary, credit analysis, financial analysis, recommendation — and 2-4 risk flags assessing where the analysis rests on fragile assumptions: a domestic content percentage close to the threshold, an energy community boundary subject to annual update, a PPA rate that hasn't been contracted yet.

Use Case

100 MW solar project, Kern County, California. CapEx: \$120M.

The developer knows the base ITC is 6%. What they don't know:

- Does PWA compliance get them to 30%? Yes — both boxes checked → 5x multiplier applies
- Is Kern County an energy community? Yes — coal closure designation per IRS Notice 2023-29 → +10%
- Does 45% domestic content clear the threshold? Yes — solar threshold is 40% → +10%
- What does that stack to and what's it worth? 40% stacked rate = \$48M credit on \$120M CapEx

Basis answers all four questions in one request, with sourcing for each determination.

Output and Results

Every analysis returns a structured memo with:

- **Credit value** — dollar amount of the IRA credit on the specific project
- **Stacked rate** — final ITC/PTC rate after all bonus adders
- **IRR with / without credits + delta** — the incremental return the IRA creates
- **NPV and payback in both scenarios**
- **Risk flags** — 2–4 items with severity ratings flagging fragile assumptions
- **Regulatory note** — anything from the news scan that could affect eligibility
- **Recommendation** — two sentences the developer can act on

Output and Results

Basis is a single-project analyzer today. The architecture extends naturally to:

- **Portfolio screening** — run 50 projects overnight, rank by credit-adjusted IRR
- **Monitoring** — re-run existing projects when new Treasury notices drop; the news agent picks up rule changes automatically
- **Tax equity structuring** — the credit value output is exactly the number a tax equity investor needs to price a deal

The broader point: multi-agent AI is well-suited to regulatory analysis problems where the rules are complex, layered, and frequently updated — and where the cost of getting it wrong is high.

Thank You